☁ Hands-On Lab

Launch an EKS Cluster from EC2 Ubuntu

Step-by-step CLI lab: install tools, configure AWS, and spin up a production-ready Kubernetes cluster on EKS.

Duration:**45–60 min** | OS:**Ubuntu 22.04 (EC2)** | ☁ Cloud:**AWS EKS** | Tool:**eksctl**

// LAB SECTIONS

01 Prerequisites & EC2 Setup

~5 min

Launch an EC2 instance that will act as your management/jump host to orchestrate the EKS cluster.

// Recommended EC2 Instance

AMI: Ubuntu 22.04 LTS | Type: t2.micro or t3.small (Free Tier eligible) | Storage: 20 GB gp2 | Security Group: Allow SSH (port 22)

⚠ **IAM permissions required:** Your EC2 instance's IAM role (or user) must have **AdministratorAccess** or at minimum: EKS full access, EC2, IAM, CloudFormation, and VPC permissions.

Update system packages first

bash copy

```
sudo apt-get update -y && sudo apt-get upgrade -y
```

Install required base tools

bash copy

```
sudo apt-get install -y curl wget unzip tar git jq
```

02 Install AWS CLI v2

~5 min

The AWS CLI is the foundation for authenticating and interacting with AWS services.

Download and install AWS CLI v2

bash copy

```
# Download the installer
curl "https://awscli.amazonaws.com/awscli-exe-linux-x86_64.zip" \
     -o "awscliv2.zip"

# Unzip and install
unzip awscliv2.zip
sudo ./aws/install

# Clean up
rm -rf awscliv2.zip aws/
```

Verify AWS CLI installation

bash copy

```
aws --version
```

✓ EXPECTED OUTPUT

```
aws-cli/2.x.x Python/3.x.x Linux/... exe/x86_64
```

03 Install kubectl

~3 min

kubectl is the Kubernetes CLI tool you'll use to interact with your EKS cluster after it's created.

Install kubectl (latest stable)

bash copy

```
# Get latest stable version string
KUBECTL_VERSION=$(curl -sL https://dl.k8s.io/release/stable.txt)

# Download kubectl binary
curl -LO "https://dl.k8s.io/release/${KUBECTL_VERSION}/bin/linux/amd64/kubectl"

# Make executable and move to PATH
chmod +x kubectl
sudo mv kubectl /usr/local/bin/kubectl
```

Verify kubectl

```bash
kubectl version --client
```

✓ EXPECTED OUTPUT

```
Client Version: v1.x.x
Kustomize Version: v5.x.x
```

04 Install eksctl

~3 min

eksctl is the official CLI for creating and managing EKS clusters. It automates CloudFormation stacks, VPC setup, and node groups.

Download and install eksctl

```bash
# Download latest eksctl
curl --silent --location \
  "https://github.com/weaveworks/eksctl/releases/latest/download/eksctl_$(uname -s)_amd64.tar.gz" \
  | tar xz -C /tmp

# Move to PATH
sudo mv /tmp/eksctl /usr/local/bin

# Verify
eksctl version
```

✓ EXPECTED OUTPUT

```
0.x.x
```

05 Configure AWS Credentials

~5 min

Two options: attach an IAM role to the EC2 instance (recommended) or configure credentials manually.

// Option A: IAM Role (Recommended)

Attach an IAM role with required permissions to the EC2 instance from the AWS Console. No keys needed — credentials auto-rotate.

// Option B: Access Keys

Create an IAM user with programmatic access, download access keys, and configure with `aws configure`.

Option B: Configure access keys manually

```bash
```

```
aws configure
```

```
INTERACTIVE PROMPT
AWS Access Key ID [None]:     <your-access-key>
AWS Secret Access Key [None]: <your-secret-key>
Default region name [None]:   ap-south-1
Default output format [None]: json
```

Verify AWS identity

bash copy

```
aws sts get-caller-identity
```

```
✓ EXPECTED OUTPUT
{
    "UserId": "AIDA...",
    "Account": "123456789012",
    "Arn": "arn:aws:iam::123456789012:user/eks-admin"
}
```

06 Create EKS Cluster

~15-20 min

Behind the scenes: eksctl creates 2 CloudFormation stacks — one for the EKS control plane (managed by AWS) and one for the managed node group (your worker EC2 nodes).

Two approaches: quick single command, or a YAML config file (recommended for reproducibility).

Method 1: Quick one-liner command

bash copy

```
eksctl create cluster \
  --name my-eks-cluster \
  --region ap-south-1 \
  --nodegroup-name standard-workers \
  --node-type t3.medium \
  --nodes 2 \
  --nodes-min 1 \
  --nodes-max 3 \
  --managed
```

Method 2: YAML config file (recommended)

Create a cluster config file for version-controlled, reproducible clusters.

bash copy

```
cat > cluster.yaml <<EOF
apiVersion: eksctl.io/v1alpha5
kind: ClusterConfig
```

```
metadata:
  name: my-eks-cluster
  region: ap-south-1
  version: "1.29"

managedNodeGroups:
  - name: ng-standard
    instanceType: t3.medium
    minSize: 1
    desiredCapacity: 2
    maxSize: 3
    volumeSize: 20
    ssh:
      allow: true
      publicKeyName: my-keypair  # replace with your EC2 key pair
    iam:
      withAddonPolicies:
        autoScaler: true
        cloudWatch: true
        albIngress: true

cloudWatch:
  clusterLogging:
    enableTypes:
      - api
      - audit
      - authenticator
EOF
```

bash copy

```
eksctl create cluster -f cluster.yaml
```

This takes 15–20 minutes. eksctl provisions the VPC, subnets, IAM roles, and EC2 worker nodes. Watch the output for CloudFormation stack events.

What eksctl creates automatically

RESOURCE	DETAILS
VPC	New VPC with public + private subnets across 2 AZs
IAM Roles	EKS cluster role + Node instance role with required policies
Security Groups	Control plane SG + node SG + cluster communication rules
EKS Control Plane	Managed Kubernetes API server (you don't manage this)
Node Group	Auto Scaling Group of EC2 instances as worker nodes
kubeconfig	Automatically updated at ~/.kube/config

07 Verify & Explore the Cluster

~5 min

Update kubeconfig (if needed)

bash copy

```
aws eks update-kubeconfig \
  --region ap-south-1 \
  --name my-eks-cluster
```

Verify nodes are Ready

bash copy

```
kubectl get nodes -o wide
```

✓ EXPECTED OUTPUT

```
NAME                                        STATUS   ROLES    AGE   VERSION
ip-192-168-x-x.ap-south-1.compute.internal   Ready    <none>   2m    v1.29.x
ip-192-168-x-x.ap-south-1.compute.internal   Ready    <none>   2m    v1.29.x
```

Explore cluster components

bash copy

```
# View all system pods (control plane components)
kubectl get pods -n kube-system

# View cluster info
kubectl cluster-info

# View all namespaces
kubectl get namespaces

# Check node resource usage (if metrics-server installed)
kubectl top nodes
```

List node groups and cluster via eksctl

bash copy

```
eksctl get cluster
eksctl get nodegroup --cluster my-eks-cluster
```

08 Deploy a Test Application

~5 min

Deploy nginx to validate the cluster is fully functional end-to-end.

Deploy nginx and expose via LoadBalancer

bash copy

```
# Create nginx deployment
kubectl create deployment nginx \
```

```
  --image=nginx:latest \
  --replicas=2

# Expose it via AWS ELB (LoadBalancer)
kubectl expose deployment nginx \
  --type=LoadBalancer \
  --port=80 \
  --name=nginx-svc

# Watch pods come up
kubectl get pods -w
```

Get the LoadBalancer URL

bash copy

```
kubectl get svc nginx-svc
```

✓ EXPECTED OUTPUT (WAIT 1-2 MIN FOR EXTERNAL-IP)

```
NAME        TYPE           CLUSTER-IP     EXTERNAL-IP                          PORT(S)        AGE
nginx-svc   LoadBalancer   10.100.x.x     abc123.ap-south-1.elb.amazonaws.com   80:3xxxx/TCP   2m
```

bash copy

```
# Test it works!
curl http://$(kubectl get svc nginx-svc -o jsonpath='{.status.loadBalancer.ingress[0].hostname}')
```

Success! If you see nginx's HTML response, your EKS cluster is fully working end-to-end with load balancing.

09 Cleanup — Delete the Cluster

~10 min

Important: EKS clusters and worker nodes incur AWS costs. Delete the cluster when you're done with the lab to avoid charges. The control plane alone costs ~$0.10/hr.

Delete test app first

bash copy

```
kubectl delete svc nginx-svc
kubectl delete deployment nginx
```

ℹ Delete the LoadBalancer service first — otherwise the AWS ELB won't be cleaned up when you delete the cluster.

Delete the entire EKS cluster

bash copy

```
eksctl delete cluster \
  --name my-eks-cluster \
  --region ap-south-1
```

Deletion takes 10–15 minutes. eksctl deletes CloudFormation stacks, node groups, VPC, and IAM roles in the correct order.

Verify deletion

bash copy

```
eksctl get cluster
aws eks list-clusters --region ap-south-1
```

✓ Lab Summary

TOOL	PURPOSE	VERSION CHECK
aws cli	AWS API access & authentication	aws --version
kubectl	Kubernetes cluster management	kubectl version --client
eksctl	EKS cluster lifecycle management	eksctl version

Next steps: Try adding the Cluster Autoscaler, install metrics-server for `kubectl top`, add the AWS Load Balancer Controller, or deploy a real app with Ingress and TLS.